



RUDRA

Finding Memory Safety Bugs in Rust
at the Ecosystem Scale

Das Problem

- Rust legt viel Wert auf Memory Safety
 - Unsafe-Code wird dennoch oft benötigt
- Häufige Fehlerquelle, selbst bei genauem Auditing!

```
pub fn allow (&mut self, irq: Irq) {  
    // IRQ_clear_mask  
    let mut irqline = irq as u8;  
    let port: &mut IoPort;  
    if irqline < 8 {  
        port = &mut self.data1;  
    } else {  
        port = &mut self.data2;  
        irqline -= 8;  
    }  
}
```

```
unsafe {  
    let value = port.inb() & !(1 << irqline);  
    port.outb(value);  
}
```

Rudra als Lösung

- Untersucht Unsafe-Codeblöcke
- Analysen mithilfe verschiedener Algorithmen
- Mehr dazu im finalen Vortrag...

```
2026-06-11 15:54:29.307814 | INFO | [rudra-progress] Running cargo rudra
2026-06-11 15:54:29.531862 | INFO | [rudra-progress] Running rudra for target lib:test-rust-package
2026-06-11 15:54:29.686785 | INFO | [rudra-progress] Rudra started
2026-06-11 15:54:29.686987 | INFO | [rudra-progress] SendSyncVariance analysis started
2026-06-11 15:54:29.687046 | INFO | [rudra-progress] SendSyncVariance analysis finished
2026-06-11 15:54:29.687075 | INFO | [rudra-progress] UnsafeDataflow analysis started
2026-06-11 15:54:29.687784 | INFO | [rudra-progress] UnsafeDataflow analysis finished
2026-06-11 15:54:29.687817 | INFO | [rudra-progress] Rudra finished
Error (SendSyncVariance:/PhantomSendForSend/NaiveSendForSend/RelaxSend): Suspicious impl of 'Send' found
→ src/lib.rs:10:1, 10:40
```

```
Warning (UnsafeDataflow:/ReadFlow): Potential unsafe dataflow issue in `map_reference`
→ src/lib.rs:13:1, 25:2
pub fn map_reference<T, F>(value: &mut T, f: F)
where
    F: Fn(T) → T,
{
    unsafe {
        // Read manually from the pointer to bypass the lifetime.
        let deref_value = ptr::read(value);
        // Invoke the mapping function, which can potentially panic and cause
        // the deref_value to become double-freed.
        let mapped_value = f(deref_value);
        ptr::write(value, mapped_value);
    }
}
```

Zwischenstand: Setup

- Projekt bereitgestellt in mehreren Docker-Images
- Basiert auf Rust 2020 und wird nicht mehr maintained
- Versions-Schwierigkeiten bei (transitiven) Dependencies
- Debian Base-Image ist veraltet/archiviert

Zwischenstand: Setup

- Projekt bereitgestellt in mehreren Docker-Images
 - Basiert auf Rust 2020 und wird nicht mehr maintained
 - Versions-Schwierigkeiten bei (transitiven) Dependencies
 - Debian Base-Image ist archiviert
-
- Alte Dependency-Versionen mussten im Lockfile „erzwungen“ werden
 - Install-Scripts und Dockerfiles wurden angepasst
 - Usability-Test war erfolgreich

Zwischenstand: Setup

```
2026-06-11 15:54:29.307814 | INFO | [rudra-progress] Running cargo rudra
2026-06-11 15:54:29.531862 | INFO | [rudra-progress] Running rudra for target lib:test-rust-package
2026-06-11 15:54:29.686785 | INFO | [rudra-progress] Rudra started
2026-06-11 15:54:29.686987 | INFO | [rudra-progress] SendSyncVariance analysis started
2026-06-11 15:54:29.687046 | INFO | [rudra-progress] SendSyncVariance analysis finished
2026-06-11 15:54:29.687075 | INFO | [rudra-progress] UnsafeDataflow analysis started
2026-06-11 15:54:29.687784 | INFO | [rudra-progress] UnsafeDataflow analysis finished
2026-06-11 15:54:29.687817 | INFO | [rudra-progress] Rudra finished
Error (SendSyncVariance:/PhantomSendForSend/NaiveSendForSend/RelaxSend): Suspicious impl of `Send` found
→ src/lib.rs:10:1: 10:40
```

```
Warning (UnsafeDataflow:/ReadFlow): Potential unsafe dataflow issue in `map_reference`
→ src/lib.rs:13:1: 25:2
pub fn map_reference<T, F>(value: &mut T, f: F)
where
    F: Fn(T) → T,
{
    unsafe {
        // Read manually from the pointer to bypass the lifetime.
        let deref_value = ptr::read(value);
        // Invoke the mapping function, which can potentially panic and cause
        // the deref_value to become double-freed.
        let mapped_value = f(deref_value);
        ptr::write(value, mapped_value);
    }
}
```

Zwischenstand: Evaluation

- Logs aus dem Paper konnten schnell reproduziert werden
- „cargo download“ ist veraltet und wurde durch ein Python-Script ersetzt
- Erhaltene Werte sind nahezu identisch zum Paper
 - Unterschiede durch veraltete URLs in Build-Scripts

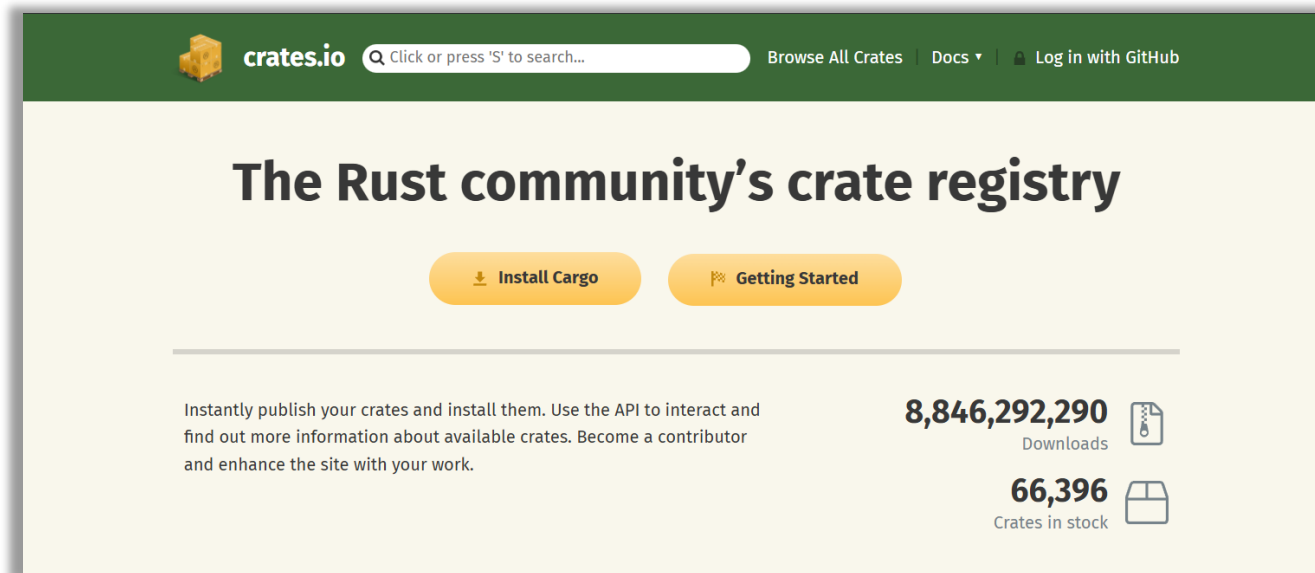
```
Bugs count for SendSyncVariance algorithm
High - visible 117 / internal 60
Med - visible 179 / internal 98
Low - visible 195 / internal 111
Bugs count for UnsafeDataflow algorithm
High - visible 65 / internal 8
Med - visible 119 / internal 17
Low - visible 163 / internal 31
```

Zwischenstand: Evaluation (STL)

- Analyse der gesamten Rust Standard Library
 - Analysiert auch eine alte Version der rustc-rayon Crate
- Kein Lockfile → Cargo lädt inkompatible Versionen runter
- Zurückspulen von Crates.io Index ist schwierig
- Lockfile mühsam durch manuelles Pinnen von Versionen erstellt

Zwischenstand: Evaluation (Crates.io)

- Alle 42.627 Crates der Package-Registry wurden für das Paper analysiert
- Benötigt viel Speicherplatz (> 256 GB) und viel Zeit (> 10 Stunden)
- Evaluations-Scripts haben teilweise nicht funktioniert
- Probleme beim File-Locking der Projekte (WSL, CachyOS)



<https://web.archive.org/web/20210813085551/https://crates.io/>

Zwischenstand: Evaluation (Crates.io)

- Evaluation hat >20 Stunden gedauert
 - auf einem Intel i9-11900H mit 32 GB RAM
- Ergebnisse stimmen fast mit 2021 überein
- 48 Compile-Fehler zurückzuführen auf:
 - Build-Scripts mit veralteten URLs
 - Hardware-Inkompatibilität (CPU zu neu)
 - Git-Commits wurden nicht fest referenziert

```
Reports for analyzer SendSyncVariance
```

```
On high: 366
```

```
On med: 791
```

```
On low: 1174
```

```
Reports for analyzer UnsafeDataflow
```

```
On high: 137
```

```
On med: 434
```

```
On low: 1214
```

```
I have no name!@318a0401ab38:/rudra-poc/paper$ █
```

Ausblick

- Weitere Experimente
- Auswirkung von Rudra auf Rust-Safety
- Statusübersicht der gemeldeten Bug-Reports
- Aktueller Stand von statischen Analysen
- Verwendung von KI für Safety-Analysen

Quellen

- Rudra Beispiel
<https://github.com/sslabs-gatech/Rudra-Artifacts>
- Rust Logo
<https://github.com/rust-lang/rust-artwork>
- Rudra: Finding Memory Safety Bugs in Rust at the Ecosystem Scale
<https://dl.acm.org/doi/10.1145/3477132.3483570>

Anhang

Evaluation (Claims)

2021

```
$ ./recreate_bugs.py
(...)
Bugs count for SendSyncVariance algorithm
High - visible 118 / internal 60
Med - visible 181 / internal 98
Low - visible 197 / internal 111
Bugs count for UnsafeDataflow algorithm
High - visible 65 / internal 8
Med - visible 119 / internal 17
Low - visible 163 / internal 31
```

```
The numbers reported in the paper:
Bugs count for SendSyncVariance algorithm
High - visible 118 / internal 59
Med - visible 182 / internal 96
Low - visible 197 / internal 109
Bugs count for UnsafeDataflow algorithm
High - visible 65 / internal 7
Med - visible 118 / internal 15
Low - visible 162 / internal 28
```

2026

```
Bugs count for SendSyncVariance algorithm
High - visible 117 / internal 60
Med - visible 179 / internal 98
Low - visible 195 / internal 111
Bugs count for UnsafeDataflow algorithm
High - visible 65 / internal 8
Med - visible 119 / internal 17
Low - visible 163 / internal 31
```

Evaluation (STL)



rudra-std-report.txt

Evaluation (Crates.io)

2021

```
{<Status.OKAY: 1>: 33223, <Status.EARLY_COMPILE_ERROR: 2>: 6656, <Status.LINT_COMPILE_ERROR: 3>: 3, <Status.EMPTY_TARGET: 4>: 1974, <Status.METADATA_ERROR: 5>: 751, <Status.ONLY_MAC_OS_ERROR: 6>: 18}
Reports for analyzer SendSyncVariance
  On high: 367
  On med: 793
  On low: 1176
Reports for analyzer UnsafeDataflow
  On high: 137
  On med: 434
  On low: 1214
```

2026

```
{<Status.OKAY: 1>: 33175, <Status.EARLY_COMPILE_ERROR: 2>: 6704, <Status.LINT_COMPILE_ERROR: 3>: 3, <Status.EMPTY_TARGET: 4>: 1974, <Status.METADATA_ERROR: 5>: 751, <Status.ONLY_MAC_OS_ERROR: 6>: 18}
Reports for analyzer SendSyncVariance
  On high: 366
  On med: 791
  On low: 1174
Reports for analyzer UnsafeDataflow
  On high: 137
  On med: 434
  On low: 1214
```


Evaluation (Crates.io)

Root Cause	Count	Crates affected
OpenBLAS: cannot determine CPU target	17	<code>argmin_core</code> , <code>dars</code> , <code>etl</code> , <code>godunov</code> , <code>linxal</code> , <code>nalgebra-lapack</code> , <code>ndarray-glm</code> , <code>ndarray-odeint</code> , <code>nymph</code> , <code>openblas-src</code> , <code>saber</code> , <code>sloword2vec</code> , <code>superlu</code> , <code>superlu-sys</code> , <code>wee-matrix</code> , <code>wyrm</code>
libsodium download: HTTP 403	11	<code>holochain*</code> (6), <code>lib3h</code> , <code>lib3h_sodium</code> , <code>purple_address_gen</code> , <code>sim2h*</code> (2)
torch-sys: HTTP 301 redirect	5	<code>nnsplit</code> , <code>rust-bert</code> , <code>sbert</code> , <code>tch</code> , <code>torch-sys</code>
intel-mkl: download failures	5	<code>intel-mkl-src</code> , <code>intel-mkl-sys</code> , <code>ndarray-vision</code> , <code>pardiso-src</code> , <code>petal-decomposition</code>
lindera-ipadic: download/corruption	4	<code>gchdb</code> , <code>lindera</code> , <code>lindera-cli</code> , <code>lindera-ipadic</code> , <code>lindera-tantivy</code>
libsodium-sys: corrupted tarball	4	<code>exonum_libsodium-sys</code> , <code>exonum_sodiumoxide</code> , <code>jwt-compact</code> , <code>pwbox</code>
caca-sys: build script assertion	2	<code>caca</code> , <code>caca-sys</code>